

DRY – Don't Repeat Yourself

DRY stands for "Don't Repeat Yourself" and it's a fundamental concept in software development. The idea is to make sure that every piece of logic is represented in one single place in the codebase. There are a few reasons for this:

- Maintainability: When code is duplicated, any change or bug fix needs to be made in multiple places, increasing the risk of errors. DRY code centralizes the logic, so updates and fixes are only made in one place.
- Reusability: Modular, DRY code is easier to reuse across different parts of a program or even in different projects.
- Readability: DRY code is cleaner and easier to read.

Implementing DRY code

- Functions: Encapsulate repeated code into functions that can be reused. Functions help to abstract away details and promote reuse.

Before DRY

```
print("Welcome, Alice!")  
  
print("You have 3 new messages.")  
  
print("Welcome, Bob!")  
  
print("You have 5 new messages.")
```

After DRY

```
def welcome_user(name, message_count):  
    print(f"Welcome, {name}!")  
    print(f"You have {message_count} new messages.")  
  
welcome_user("Alice", 3)  
welcome_user("Bob", 5)
```

- Classes: Use classes to define common behaviors and attributes for objects.

This allows for creating reusable and modular code.

Before DRY

```
car1_brand = "Toyota"  
car1_model = "Camry"  
car1_year = 2018  
car1_color = "Blue"
```

```
car2_brand = "Honda"  
car2_model = "Accord"  
car2_year = 2020  
car2_color = "Red"
```

After DRY

```
class Car:  
    def __init__(self, brand, model, year, color):  
        self.brand = brand  
        self.model = model
```

```
self.year = year
```

```
self.color = color
```

```
car1 = Car("Toyota", "Camry", 2018, "Blue")
```

```
car2 = Car("Honda", "Accord", 2020, "Red")
```

- Modules: Organize code into modules to promote reuse across different parts of a program. Modules encapsulate related code into a single file, making it easier to manage and reuse.

```
# my_module.py
```

```
def greet(name):
```

```
    return f"Hello, {name}!"
```

```
def add(a, b):
```

```
    return a + b
```

```
# main.py
```

```
import my_module
```

```
print(my_module.greet("Alice"))
```

```
print(my_module.add(5, 3))
```

- Inheritance: Use inheritance in object-oriented programming to share common functionality among related classes, avoiding duplication.

```
class Animal:
```

```
    def __init__(self, name):
```

```
self.name = name
```

```
def speak(self):
```

```
    raise NotImplementedError("Subclasses must implement this method")
```

```
class Dog(Animal):
```

```
    def speak(self):
```

```
        return f'{self.name} says 'Woof''
```

```
class Cat(Animal):
```

```
    def speak(self):
```

```
        return f'{self.name} says 'Meow''
```

```
my_dog = Dog("Rex")
```

```
my_cat = Cat("Whiskers")
```

```
print(my_dog.speak())
```

```
print(my_cat.speak())
```